KnowledgeBase AI — Implementation Manual

6-Phase Architecture Blueprint, Engineering Rationale & Verification

Version 1.1 • Go + Fiber v2 + PostgreSQL + pgvector + Redis + Asynq + Gemini API

1. Executive Summary & Purpose

KnowledgeBase AI is an enterprise-grade, multi-tenant Retrieval-Augmented Generation (RAG) backend. Unlike trivial AI wrappers that simply forward uploaded documents directly into prompts, KnowledgeBase AI is engineered for production robustness, strict tenant isolation, asynchronous durable ingestion, recoverable background pipelines, and verifiable citation lineage.

This specification organizes the complete lifecycle of the platform into 6 sequential phases punctuated by 5 concrete checkpoints. Each phase addresses specific engineering failure modes, enforces strict interface boundaries, and guarantees database invariants before proceeding.

> **Core Architectural Principles**
> 1. Asynchronous Ingestion: The HTTP API never performs embedding generation or document parsing inline.
> 2. Durable Outbox Pattern: Ingestion jobs and outbox events are committed transactionally with document metadata.
> 3. Strict Multi-Tenancy: Every single database query and vector search is scoped to the authenticated organization.
> 4. Untrusted Model Output: LLM responses cannot bypass authorization or invent fabricated citation IDs.
> 5. Process Isolation: Fiber API and Asynq background workers share the codebase but run as isolated services.

2. Technology Stack & Component Matrix

Layer	Technology	Design Purpose / Rationale
Language	Go 1.22+	Memory efficiency, compile-time type safety, exceptional concurrency.
HTTP Framework	Fiber v2 (fasthttp)	High-performance routing; safe context isolation for background jobs.
Primary Database	PostgreSQL 16	ACID transactional state, outbox events, and user/org ownership.
Vector Search	pgvector (HNSW)	Cosine similarity search over 768-dimensional Gemini embeddings.
SQL Access	pgx/v5 (pgxpool)	Direct parameterized queries without ORM latency overhead.
Task Queue	Redis + Asynq	Durable task retry policies, uniqueness deduplication, priority queues.
Authentication	JWT + Refresh Sessions	15-minute access JWTs; rotating hashed refresh tokens with family revocation.
AI Provider	Gemini API (1.5 Pro / Embeddings)	Encapsulated provider interface requesting 768-dim embeddings.

3. The 6-Phase Breakdown & 5 Checkpoints

To deliver a reliable backend without accumulating technical debt, implementation proceeds strictly across 6 sequential phases. Transitions between phases are guarded by 5 explicit verification checkpoints.

Phase	Scope & Description	Checkpoint Gate	Status
Phase 1	Repository Foundation & Infrastructure	Checkpoint 1: Compiling binaries, Fiber router, migrations, health probes	COMPLETED
Phase 2	Identity, Authentication & Tenancy	Checkpoint 2: Argon2id, JWT/refresh rotation, org RBAC, cross-tenant tests	COMPLETED
Phase 3	Document Uploads & Outbox Pipeline	Checkpoint 3: Streaming upload, SHA-256, atomic storage, outbox persistence	PLANNED
Phase 4	Asynq Worker & Vector Ingestion	Checkpoint 4: Dispatcher, PDF/TXT/MD parsers, chunking, pgvector writes	PLANNED
Phase 5	Vector Search & RAG Chat Engine	Checkpoint 5: Cosine search, Gemini generation, verified citations, SSE streaming	PLANNED
Phase 6	Hardening, Observability & Evaluation	Final Release: Rate limits, Prometheus metrics, E2E tests, AI benchmark suite	PLANNED

Detailed Phase Analysis: What, How, and For What Purpose

Phase 1: Repository Foundation & Core Infrastructure (Checkpoint 1) — COMPLETED

- **What:** Establishes Go module structure, Fiber v2 router, configuration loader, PostgreSQL & Redis connection pools, database migrations, and health check endpoints.
- **How:** Uses pgxpool with connection limits, go-redis client, standard error envelopes, and a multi-stage Dockerfile running non-root.
- **Purpose:** Provides a rock-solid, production-ready substrate ensuring external services can be probed and scaled independently.

Phase 2: Identity, Multi-Tenancy & Authentication (Checkpoint 2) — COMPLETED

- **What:** User registration, Argon2id password hashing, login, rotating opaque refresh tokens, JWT access tokens, organization lifecycle, and RBAC memberships (owner, admin, member).
- **How:** Argon2id with OWASP parameters (64MB memory, 1 iteration, 4 parallelism); 15m JWT access tokens; cryptographically random refresh tokens stored as SHA-256 hashes in PostgreSQL; family_id reuse revocation; single-transaction organization and owner creation; route-level RequireOrgMembership middleware.
- **Purpose:** Enforces absolute tenant isolation, prevents unauthorized cross-organization data access, and eliminates session hijacking risks.

Phase 3: Document Management & Ingestion Outbox (Checkpoint 3)

- **What:** Streaming multipart uploads, SHA-256 deduplication, file validation (PDF, TXT, MD), and transactional ingestion records.
- **How:** Streams raw bytes directly to storage without buffering in memory; inserts document record, ingestion job, and outbox event inside a single PostgreSQL transaction.
- **Purpose:** Prevents memory exhaustion on large file uploads and eliminates race conditions between storage and database state.

Phase 4: Asynq Ingestion Worker & Vector Ingestion (Checkpoint 4)

- **What:** Background task worker consuming outbox events, document parsing, text chunking, and vector embedding generation.
- **How:** Polls unpublished outbox events using 'FOR UPDATE SKIP LOCKED', enqueues deterministic tasks into Redis Asynq, calls Gemini embeddings (768 dimensions), and writes chunks to pgvector with HNSW indexing.
- **Purpose:** Decouples heavy file parsing and AI API latencies from HTTP response times while guaranteeing at-least-once processing.

Phase 5: Retrieval-Augmented Generation & Chat Engine (Checkpoint 5)

- **What:** Semantic vector search, context prompt engineering, Gemini chat generation, and Server-Sent Events (SSE) streaming.
- **How:** Computes query embeddings, queries pgvector strictly scoped by organization_id with cosine similarity and relevance thresholds, injects retrieved passages into guarded prompts, and validates citation chunk IDs against the database.
- **Purpose:** Eliminates LLM hallucinations by enforcing citations linked strictly to retrieved, authenticated passages.

Phase 6: Production Hardening, Observability & Evaluation (Final Release)

- **What:** Atomic Redis rate limiting, Prometheus metrics, structured request tracing, comprehensive integration tests, and RAG evaluation benchmarks.
- **How:** Redis sliding-window rate limiters per org/user/endpoint, automated test suites, and RAG benchmark scoring for faithfulness, recall, and latency.
- **Purpose:** Guarantees production stability under load and ensures predictable LLM cost and accuracy bounds.

4. Phase 2 Implementation & Security Engineering

Phase 2 establishes the identity and security backbone of KnowledgeBase AI. All identity constructs, password security algorithms, token rotation lifecycles, and multi-tenant RBAC boundaries have been fully implemented.

- **Argon2id Password Security:** Complies with OWASP password storage recommendations. Uses 64MB memory, 1 iteration, 4 parallel threads, 16-byte random salt, and 32-byte key length. Verified with constant-time equality checks.
- **Dual-Token Lifecycle:** Short-lived (15-minute) HMAC-SHA256 JWT access tokens containing user_id and email claims. Opaque 32-byte refresh tokens stored exclusively as SHA-256 hashes in PostgreSQL.
- **Session Family Reuse Detection:** Each refresh chain shares a unique family_id. When an already-rotated or revoked token is presented, the system revokes the entire session family immediately, thwarting token theft attempts.
- **HttpOnly Cookie Delivery:** In accordance with TDD Section 5.2, refresh tokens are never returned in JSON response bodies or stored in localStorage. They are delivered via HttpOnly, Secure, SameSite=Lax cookies with path /api/v1/auth.
- **Single-Transaction Organization Provisioning:** POST /api/v1/organizations creates both the organization entity and the creator's owner membership within a single atomic database transaction.
- **Tenant Isolation & Role Weights:** RequireOrgMembership middleware extracts :orgID, parses UUID, verifies membership from DB, and enforces role hierarchies (owner:3, admin:2, member:1). Prevents cross-tenant access with 403 Forbidden.
- **Last-Owner Invariant:** Explicit business rule and test preventing an organization from deleting or unassigning its last remaining owner.

5. Checkpoint 2 Verification Report

Checkpoint 2 Verification Results: ALL TESTS PASSED

 Password Hashing: Argon2id generation and verification succeeded (0.15s).
 JWT Validation: Signatures, expiration limits, and tamper detection confirmed (0.10s).
 Registration & Conflict: Case normalization and 409 duplicate email conflict verified.
 Login & Cookie Security: Returned 200 OK, issued JWT, and set HttpOnly refresh cookie.
 Token Rotation: /api/v1/auth/refresh rotated token and issued fresh JWT access token.
 Family Reuse Detection: Presenting old refresh token revoked entire family (401 Unauthorized).
 Cross-Tenant Isolation: User B forbidden (403) from accessing User A's organization.
 RBAC Enforcement: Members prohibited from inviting users; admins granted privileges.
 Invariant Protection: Removal of the final owner rejected with 400 Bad Request.

6. Repository Structure in 'Knowledge AI'

The project layout adheres to clean architecture principles with clear domain separation:

Directory / File	Purpose
cmd/api/main.go	HTTP entrypoint initializing Fiber v2, repositories, auth routes, and health probes.
cmd/worker/main.go	Background process initializing Asynq server, Redis client, and graceful shutdown.
cmd/migrate/main.go	Database migration CLI supporting 'up', 'down', and 'version' commands.
internal/apperror/errors.go	Centralized standard error envelope (TDD §5.1) and domain error codes.
internal/apperror/context.go	Context extraction utilities for authenticated user_id and tenant boundaries.
internal/auth/password.go	OWASP-compliant Argon2id password hashing and constant-time verification.
internal/auth/jwt.go	HMAC-SHA256 JWT access token minting and signature validation.
internal/auth/token.go	Cryptographic random token generation and SHA-256 bytea hashing.
internal/auth/service.go	Authentication workflows: signup, login, family rotation, and logout.
internal/auth/handler.go	HTTP endpoints for /api/v1/auth with HttpOnly refresh cookie management.
internal/auth/middleware.go	JWTMiddleware validating Bearer tokens and setting context locals.
internal/identity/models.go	Domain entities: User, Organization, Membership, RefreshSession.
internal/identity/repository.go	Repository interface for persistent and test storage.
internal/identity/postgres_repo.go	PostgreSQL implementation utilizing pgxpool.Pool.
internal/identity/memory_repo.go	Thread-safe in-memory repository for unit and integration testing.
internal/identity/service.go	Organization creation, member management, and last-owner protection.
internal/identity/handler.go	HTTP endpoints for /api/v1/organizations with RBAC controls.
internal/identity/middleware.go	RequireOrgMembership middleware enforcing tenant isolation.
tests/unit/auth_test.go	Unit tests for password hashing, JWTs, rotation, and reuse revocation.
tests/unit/tenancy_test.go	Unit tests for multi-tenancy, cross-tenant denial, and RBAC roles.

7. Instructions for Pushing to GitHub

Phase 2 is complete, tested, and verified against Checkpoint 2. Run the following git commands inside '/home/lucifer/Knowledge AI' to push the updates to your repository:

- **Step 1:** Stage all new and updated files: git add .
- **Step 2:** Commit Phase 2 changes: git commit -m 'feat(phase-2): identity, multi-tenancy, and authentication with token rotation'
- **Step 3:** Push to remote repository: git push origin main